

samf.sh/talks/2026/10/21

Argonne NATIONAL LABORATORY

Why this matters at PyTorch Conference

ALCF	DOE leadership computing — runs `Aurora` + `Polaris` for open science
Aurora	`10,624` nodes · `63,744` tiles of Intel XPU — up before `torch.xpu` existed
This talk	Every team picked PyTorch anyway. How we closed the gap



Fig: *Aurora's HPE Cray EX rows at the ALCF — 10,624 nodes, 63,744 tiles. Photo: Argonne / ALCF.*

Argonne NATIONAL LABORATORY

The arc

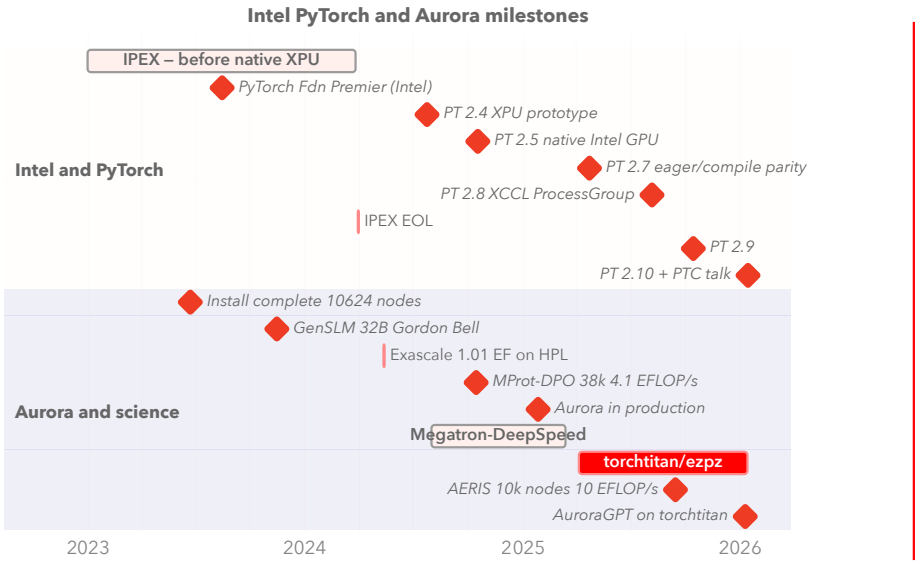
Act	Question	Reusable pattern
A · Needed	What does a first-class XPU backend actually require?	The 3 breaks we fixed + the ladder that catches the next one
B · Accomplished	How do you leave a Megatron-DeepSpeed fork for composable PyTorch?	The 50-step gate + the 768-rank proof
C · Remains	What keeps a 14-day job alive at 60k XPUs (half Aurora) → 120k tiles?	Watchdog + bad-node quarantine + layered restarts

Why Aurora needed a real backend

Workload	What it taught us
GenSLM – viral genomes	Portability before Aurora existed (Polaris + Cerebras → Gordon Bell)
MProt-DPO – protein design	38,400 ranks · 4.11 EFLOP/s at 3,200 nodes
AERIS – weather diffusion	10,080 nodes · 10.21 EFLOP/s · 90-day stable rollouts
AuroraGPT – foundation LLM	7.77T-token MDS reference → torchtitan

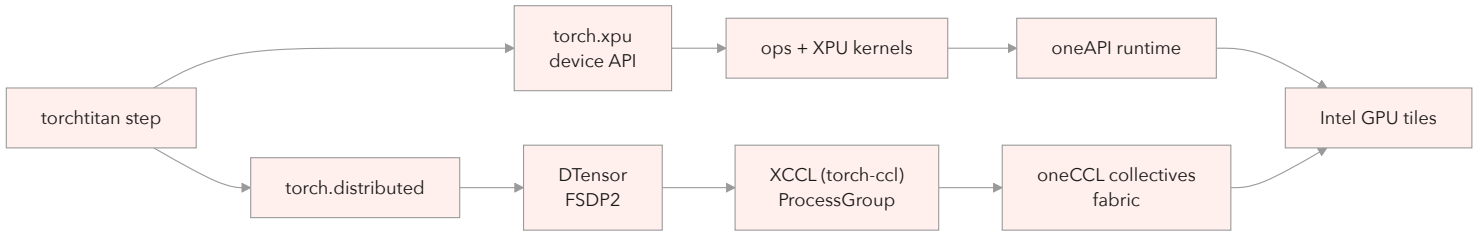
Aurora is **10,624 nodes · 63,744 tiles** – ~120k XPUs at 2 tiles/GPU.[1] `ezpz` : same training program on `Intel`, `NVIDIA`, `AMD`.[1]

When the backend landed



Intel and PyTorch – IPEX → PT 2.4–2.10+XPU (4 releases) · **Aurora and science** – 10k nodes → 120k-XPU workloads
From ezpz: distributed PyTorch across any hardware plus verified Aurora dates – on desktop the two lanes are the chart background; on phones scroll →.

A first-class XPU backend – the actual surface



Every edge is a place a job can pass a unit test and still die at scale. `IPEX` shims → native `torch.xpu` + `torch-ccl` / `XCCL` + `oneCCL` is the migration behind this diagram.

Three XPU breaks – same pattern every time

Symptom	Root cause	What we run now
`torch 2.13`: `expected DTensor, got plain tensor` on FSDP wrap	Missing pytorch#181519	Fixed in `2.14.0+xpu` · A/B on Sunspot job `12477670`
`DeviceMesh` → `does not support splitting`	`ProcessGroupXCCL` inherits `supportsSplitting() == false` (see pytorch#186548)	Local shim · pytorch#186548
`train_timeout_seconds` is a no-op	`_set_pg_timeout` has no `xpu` branch – `XCCL` stores the timeout, never arms it	Watchdog heartbeat (Act C) + two-PR upstream

Pattern: **works on CUDA, inherits a silent default on XPU.**

Why the fork broke

Megatron-DeepSpeed fork [2]	`torchtitan` + `DTensor` + `FSDP2`
`2B` to `7.77T` tokens shipped on the fork (Aug talk)	Same run continues on `torchtitan` — `4.67T` at `256` → `512` nodes
Parallelism baked into the fork	`DTensor` + `FSDP2` as composable primitives
Every PT release → rebase + patch audit	XPU integration in upstreamable layers · `torchtitan/ezpz`

The fork didn't fail scientifically. It failed operationally — it couldn't carry the XPU work without re-patching every release.

One training program across vendors

Hello world:

```python
# example.py
import ezpz
rank = ezpz.setup_torch()
world_size = ezpz.get_world_size()
dev = ezpz.get_torch_device()      # {xpu, cuda, cpu, ...}

# FSDP2 by default — accepts DeviceMesh
model = ezpz.wrap_model(torch.nn.Linear(128, 10).to(dev))

print(f"Hello from {rank}/{world_size}!")
```

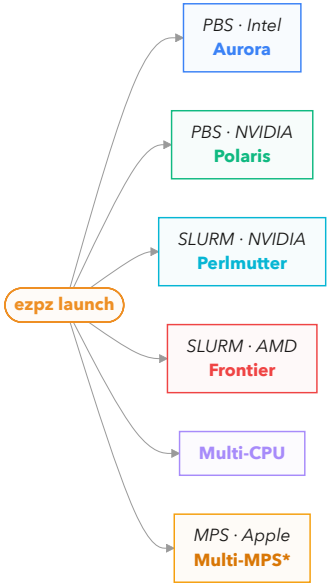
```bash
$ ezpz launch python3 example.py
Hello from rank 0/24!
Hello from rank 1/24!
# ... etc
```

Other examples:

```bash
# your own script — no changes needed
ezpz launch python3 your_training_script.py

# full HF model + dataset via ezpz examples
ezpz launch python3 -m ezpz.examples.fsdp_tp \
    --dataset=allenai/olmo-mix-1124
    --model=meta-models/Muse-Spark-Open-816B-A42B \
    --local-batch-size=1
    # ...etc
```

Same command on every machine — `ezpz` picks the backend.



* `mccl` — *mps-ddp/mccl:* `MPS` *on Apple silicon —* `ezpz launch` *even on a laptop.*

Benchmark the path the job actually uses

1. Freeze build + model + batch + precision + node count
2. Run the **training step** (not a micro-kernel) on each backend
3. Measure step time · tokens/s · collective time · memory
4. Scale node count; **remeasure after every kernel or collective change**

> A passing unit test proves an op exists. A repeatable step proves it's useful at scale.

The `XCCL` timeout fix that Act C's ops story depends on, the `XPU` / `XCCL` scaling ladder at `Sunspot`, and the watchdog that saved `job 8505298` all passed this way.[3]

What moved: MDS → `torchtitan`

Layer	Before (MDS fork + `ezpz`)	After (`torchtitan/ezpz`)
Parallelism	Custom tensor-parallel + pipeline in fork	`FSDP2` + `DTensor` sharding
Collectives	DeepSpeed's `NCCL` assumptions	`XCCL` (`torch-ccl`) → `oneCCL`
Kernels	Fork-local fused kernels	Upstreamable `XPU` kernels via `torch.xpu` – `layernorm+residual` moved; `attn` / `MLP` / `optimizer` stay in remaining-work [4]
Launch	Already `ezpz` – scheduler → nodes → `MPI`	Same `ezpz` [5] – `launch` + `checkpoint-restart`
Checkpoint	`DeepSpeed` engine save	`DCP` (`torch.distributed.checkpoint`)

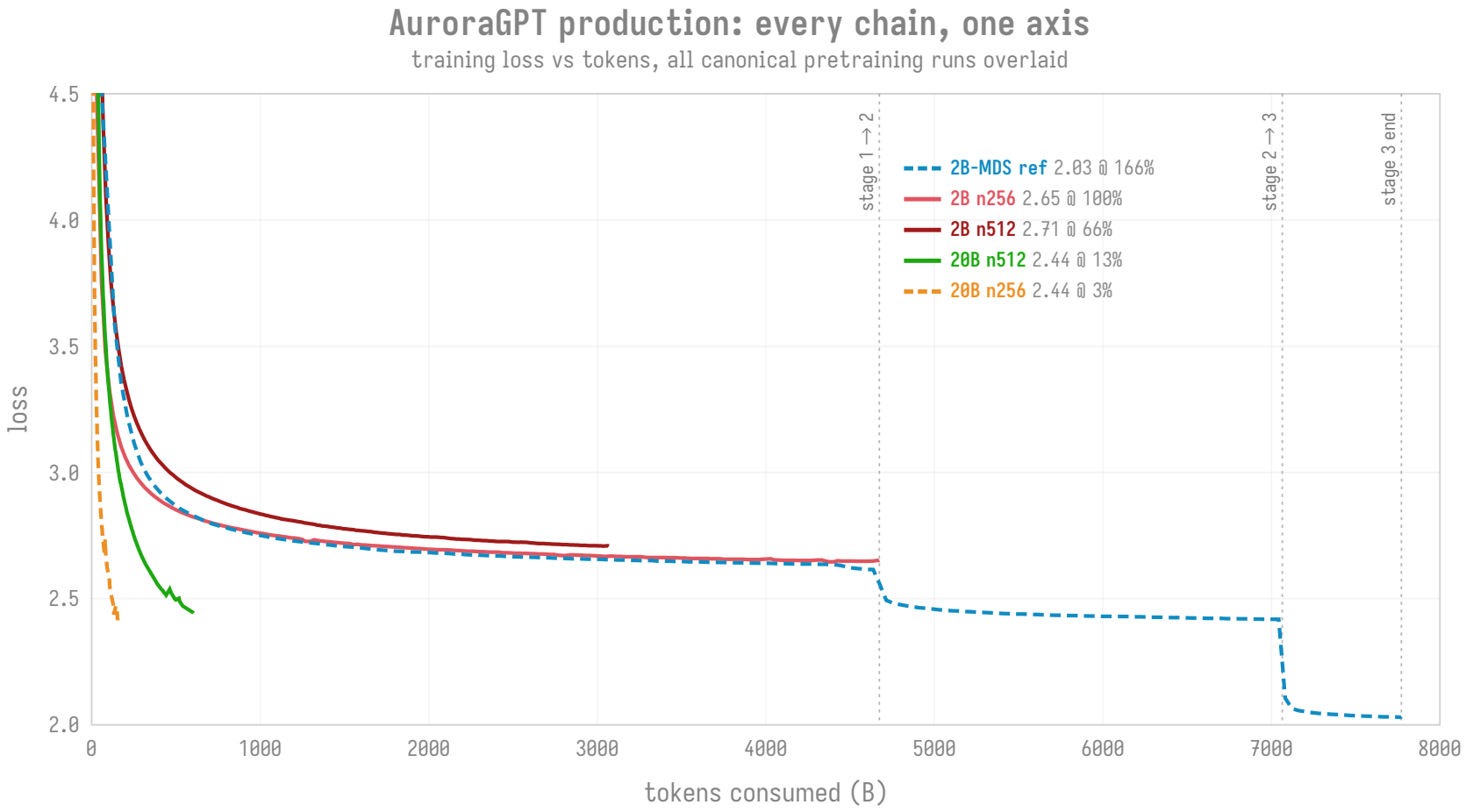
Same tokens, different owner per layer.

Testing at every layer

Layer	Gate	Link
Numerics	50-step loss – `±0.10` on final + last-10 mean	Production loss figure
Collectives	XCCL ladder `48 → 768` ranks, value-checked	`ezpz/test_dist.py`
Parallelism	TP1 / TP2 / MoE on Aurora; parity on Sunspot	`experiments/ezpz`
Portability	One gate on Aurora + Sunspot + Polaris + Perlmutter	`ezpz`
Orchestration	Failover harness – 9 fixtures	Failover writeup

`±0.10` sits just above `~0.05` shuffle noise from the streaming loader – catches bad gradients without flaking on data order.

Does the new stack train? Production says yes.



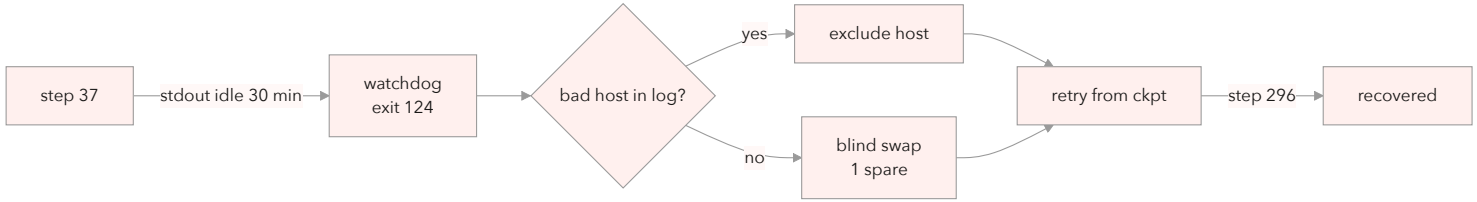
`torchtitan` 2B at 256 and 512 nodes both hit **4.674T** tokens. Not a benchmark — a continuation of the 7.77T `MDS` reference with a new mix.

For more on the trials and tribulations of TorchTitan on Aurora, see *Pre-Training LLMs on a Supercomputer* — ATPESC August 2026 · `torchtitan/ezpz` experiments · `ezpz`

Argonne NATIONAL LABORATORY

A healthy process can still be a dead job

Job `8505298` – step 37, then silence. No traceback. Watchdog at 30 min (`exit 124`), wrapper swapped a node, relaunched. Attempt 2 reached step 296.



`_set_pg_timeout` has no `xpu` branch – timeout is stored, never enforced.[6]

Failure taxonomy at ~120k XPUs

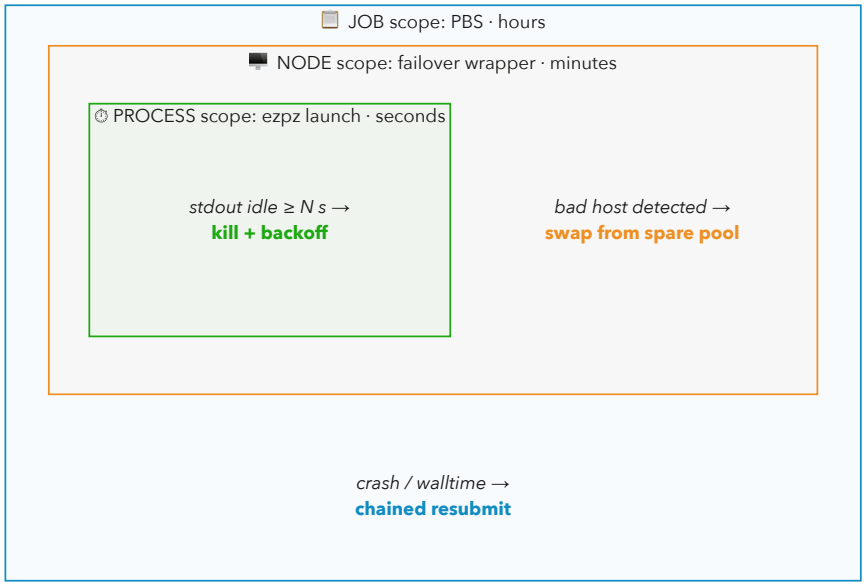
Mode	Signal	What helps	What doesn't
Silent hang	Stdout stalls, no error	Watchdog on log progress	`_set_pg_timeout` (`xpu` never arms)
Lost collective	Rank times out in `XCCL`	Node exclude + retry	Retrying on the same host
Crash / OOM	Traceback + exit code	Process restart from `ckpt`	Scheduler resubmit alone
Stale FS / DCP	Checkpoint read stalls	Async save + local staging	Sync save on shared `FS` at 10k ranks

At this scale, **every job is a test of the recovery path**.[7]

Layered restarts – the actual system

Layer	Scope	Trigger	Cost of being wrong
Process	1 rank	Crash / OOM	Tokens since last ckpt
Node	12 XPUs	Hang on one host	1 node's work + re-shard
Allocation	10k nodes	Bad FS / scheduler	Whole-job walltime

`ezpz` handles the outer two: `checkpoint-restart` + node exclude + scheduler resubmit. DCP handles the inner save.
Long jobs are not an ops concern bolted on – they **are** the product. Design them first.



Nested recovery scopes – adapted from the August talk (#32). Inner loops catch most failures; outer loops catch the rest.

What remains

Layer	Gap	What's needed
Kernels	Fused attn + MLP + optimizer across XPU	A composable kernel story that survives `torch.compile`
Collectives	Tuning, diagnostics, **real timeouts**	`XCCL` timeout enforcement + `DTensor` split fix upstream
Checkpoints	Async `DCP` at 10k ranks without `FS` collapse	Local staging → async flush
Operations	Bad-node detection before walltime burns	Heartbeat at the `ProcessGroup` level, not stdout `grep`

Bringing up the next accelerator: **test it with a restartable training job** — not isolated kernels.

What we would do again

1. **Backend + distributed together:** `torch.xpu` without `XCCL` is a demo, not a trainer.
2. **One program, many launches:** `ezpz` owns the schedule, the model doesn't branch.
3. **A science baseline that survives the migration:** 7.77T → 4.674T on the same tokens axis.
4. **Measure the step, not the kernel:** the whole training step, at the job's real scale.
5. **Recovery before throughput:** checkpoint + watchdog + quarantine before you tune.

GenSLM · MProt-DPO · AERIS · AuroraGPT — not case studies, but the workloads that forced each layer.
Patterns for running PyTorch at scale on a new accelerator.

Links and further reading

Stack	Link
`ezpz`	Launch · Checkpoint · All docs
`torchtitan/ezpz`	Experiments · Failover writeup
PyTorch XPU	`torch.xpu` · `DTensor` · `FSDP2` · DCP
Distributed	`torch-ccl` / XCCL · `oneCCL` · torch.distributed
System	Aurora · ALCF support – Aurora · ALCF news – 120k-XPU scale
Science	AuroraGPT companion · AERIS · MProt-DPO · GenSLM

Elsewhere by Sam: ezpz: distributed PyTorch across any hardware (source chronology for the timeline) · Running 50k Python processes on Aurora with `ezpz yeet` · *Pre-Training LLMs on a Supercomputer* – ATPESC Aug 2026 · BlendCorpus + TorchTitan @ ALCF · *Upcoming:* SC publication on the AuroraGPT training system

All links are in the deck at `samf.sh/talks/2026/10/21` .

Acknowledgements

This work used resources of the **Argonne Leadership Computing Facility**, a DOE Office of Science user facility at Argonne National Laboratory, supported under contract **DE-AC02-06CH11357**.

Aurora was built in partnership with **HPE** and **Intel** — Intel Max 1550 XPU (`PVC`), `oneAPI` / `oneCCL` / `XCCL` , and the upstream `torch.xpu` work tracked in Act A. Thanks to the ALCF operations and support teams, the `PyTorch` and `torchtitan` maintainers, and the science teams behind `GenSLM` , `MProt-DPO` , `AERIS` , and `AuroraGPT` .

Facility	ALCF · Aurora · Polaris
Partners	HPE · Intel (Max 1550 / `oneAPI` / `oneCCL`) · PyTorch Foundation
Teams	`GenSLM` · `MProt-DPO` · `AERIS` · `AuroraGPT` — see Links for papers

Want to run on Aurora?

Aurora and Polaris are **open to any researcher** through DOE's allocation programs — not just Argonne staff.

Program	Best for	Cadence
INCITE	Large, peer-reviewed science campaigns	Annual
ALCC	DOE-mission-aligned projects	Annual
Director's Discretionary (DD)	Porting, benchmarking, getting started — **start here**	Rolling, weeks

If you'd like to get access, **reach out** — happy to help scope a DD proposal or connect you with ALCF staff. Find me after the talk or at `foremans@anl.gov` / @saforem2.

ALCF allocations overview · DD program details · ALCF support center